

tf.searchsorted Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/searchsorted

Searches for where a value would go in a sorted sequence.

Function Signatures

```
tf.searchsorted( sorted_sequence, values, side='left',  
out_type=tf.dtypes.int32, name=None )
```

Code Examples

```
tf.searchsorted(  
sorted_sequence,  
values,  
side='left',  
out_type=tf.dtypes.int32,  
name=None  
)
```

```
tf.searchsorted(  
sorted_sequence,  
values,  
side='left',  
out_type=tf.dtypes.int32,  
name=None  
)
```

```
tf.dtypes.int32
```

```
edges = [-1, 3.3, 9.1, 10.0]  
values = [0.0, 4.1, 12.0]  
tf.searchsorted(edges, values).numpy()  
array([1, 2, 4], dtype=int32)
```

```
edges = [-1, 3.3, 9.1, 10.0]
```

```
values = [0.0, 4.1, 12.0]
```

```
tf.searchsorted(edges, values).numpy()
```

```
array([1, 2, 4], dtype=int32)
```

Documentation

Searches for where a value would go in a sorted sequence.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.searchsorted`

This is not a method for checking containment (like `python in`).

The typical use case for this operation is "binning", "bucketing", or "discretizing". The values are assigned to bucket-indices based on the edges listed in `sorted_sequence`. This operation returns the bucket-index for each value.

The side argument controls which index is returned if a value lands exactly on an edge:

The axis is not settable for this operation. It always operates on the innermost dimension (`axis=-1`). The operation will accept any number of outer dimensions. Here it is applied to the rows of a matrix:

Returns

`## Raises`